Interfețe funcționale

MAP – Cursul 5 · Java 8+ · pachetul `IF`

Definiție

- **Interfață funcțională** = interfață cu **o singură metodă abstractă**. Metodele `default` și `static` nu se numără.
- `@FunctionalInterface` nu e obligatorie: cere compilatorului să verifice regula.
- O implementăm fără clasă cu nume: prin **lambda** sau **referință la metodă**.

De la clasă anonimă la lambda

```
@FunctionalInterface
interface Operatie { double calculeaza(double a, double b); }

Operatie op1 = new Operatie() {                        // clasă anonimă
    public double calculeaza(double a, double b) { return a * b; } };
Operatie op2 = (a, b) -> a * b;                        // lambda
Operatie op3 = Math::max;                              // referință la metodă
```

Lambda

- Sintaxă: `x -> expr`, `(a, b) -> expr`, `() -> { ...; return r; }`
- Tipul vine din context (*target type*): aceeași lambda poate fi `Predicate` sau `Function`. Din acest motiv, `var f = x -> x;` nu compilează.
- Variabilele locale folosite în lambda trebuie să fie *effectively final*. Câmpurile clasei se pot modifica.
- În lambda, `this` este obiectul clasei în care e scrisă. Într-o clasă anonimă, `this` este obiectul anonim.

Interfețe predefinite (java.util.function)

Interfață	Metodă	Formă	Exemplu	Metode utile
`Predicate<T>`	`test`	T → boolean	`s -> s.getMedia() >= 5`	`and, or, negate, Predicate.not`
`Function<T,R>`	`apply`	T → R	`Integer::valueOf`	`andThen, compose`
`Consumer<T>`	`accept`	T → void	`System.out::println`	`andThen`
`Supplier<T>`	`get`	() → T	`ArrayList::new`, `LocalDate::now`	–
`UnaryOperator<T>`	`apply`	T → T	`String::toUpperCase`	caz particular de Function
`BinaryOperator<T>`	`apply`	(T, T) → T	`Integer::sum`	`maxBy, minBy`
`BiFunction<T,U,R>`	`apply`	(T, U) → R	`(a, b) -> a + b`	`andThen`
`Comparator<T>`	`compare`	(T, T) → int	`Comparator.comparing(Student::getNume)`	`thenComparing, reversed`

Referințe la metode

Tip	Referință	Lambda echivalentă
metodă statică	`Integer::valueOf`	`s -> Integer.valueOf(s)`
metoda unui obiect anume	`System.out::println`	`x -> System.out.println(x)`
metoda unui obiect oarecare	`Student::getNume`	`s -> s.getNume()`
constructor	`ArrayList::new`	`() -> new ArrayList<>()`

Unde le folosim

```
lista.forEach(System.out::println);                    // Consumer
lista.removeIf(s -> s.getMedia() < 5);                 // Predicate
lista.sort(Comparator.comparing(Student::getMedia).reversed());   // Comparator
lista.stream().filter(...).map(...)                    // Stream API: Predicate, Function
```

> Contează forma metodei (parametri → rezultat), nu numele ei: orice metodă cu forma potrivită poate fi transmisă ca referință.